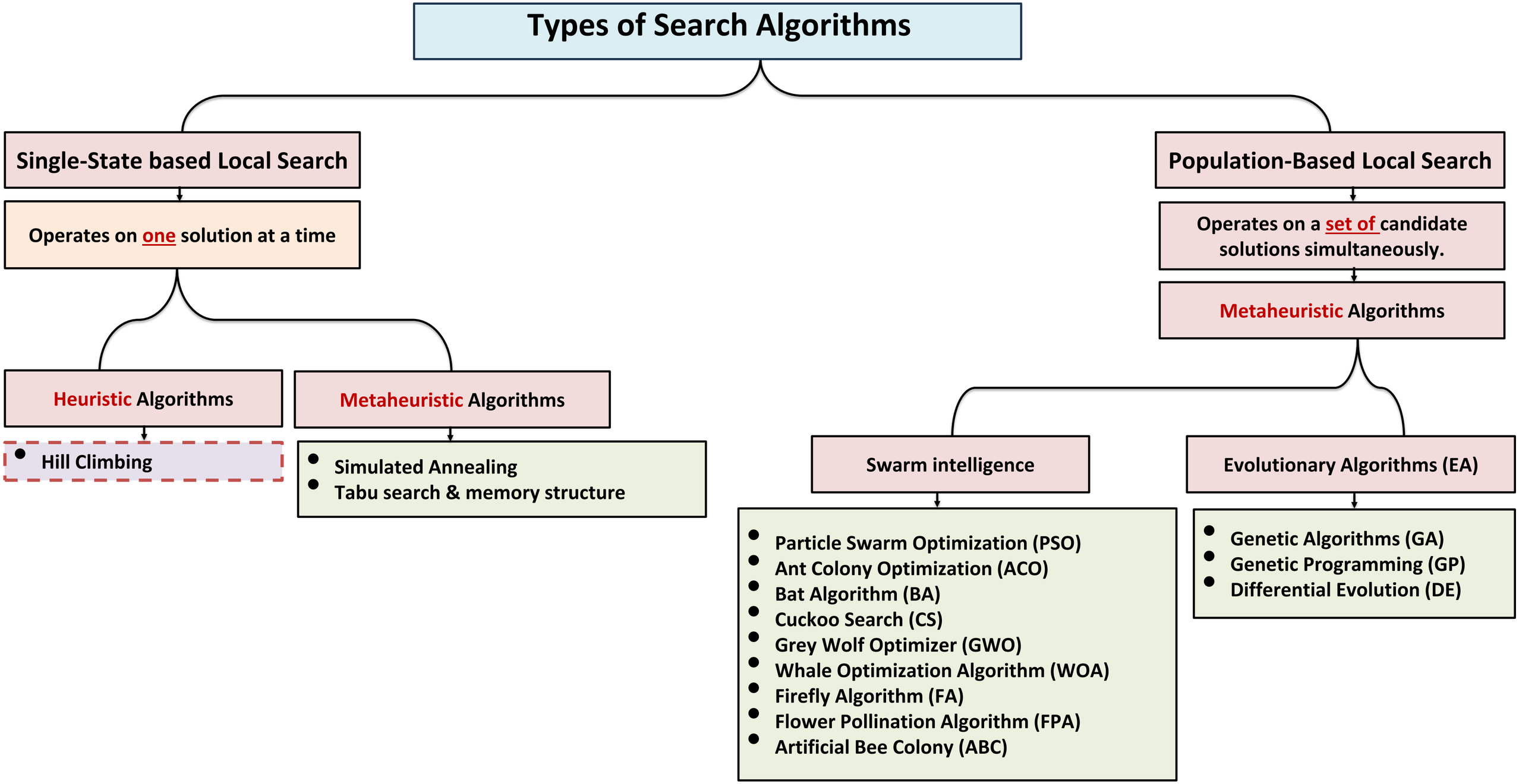
Nature Inspired Computation
DSAI 403

Assoc. Prof. **Mohamed Maher Ata**
Zewail city of science, technology, and innovation
momaher@zewailcity.edu.eg


Types of Search Algorithms
Single-State based Local Search
Operates on one solution at a time
Heuristic Algorithms
Hill Climbing
Metaheuristic Algorithms
Simulated Annealing
Tabu search & memory structure
Population-Based Local Search
Operates on a set of candidate solutions simultaneously.
Metaheuristic Algorithms
Swarm intelligence
Particle Swarm Optimization (PSO)
Ant Colony Optimization (ACO)
Bat Algorithm (BA)
Cuckoo Search (CS)
Grey Wolf Optimizer (GWO)
Whale Optimization Algorithm (WOA)
Firefly Algorithm (FA)
Flower Pollination Algorithm (FPA)
Artificial Bee Colony (ABC)
Evolutionary Algorithms (EA)
Genetic Algorithms (GA)
Genetic Programming (GP)
Differential Evolution (DE)

Hill Climbing as a simple local search algorithm

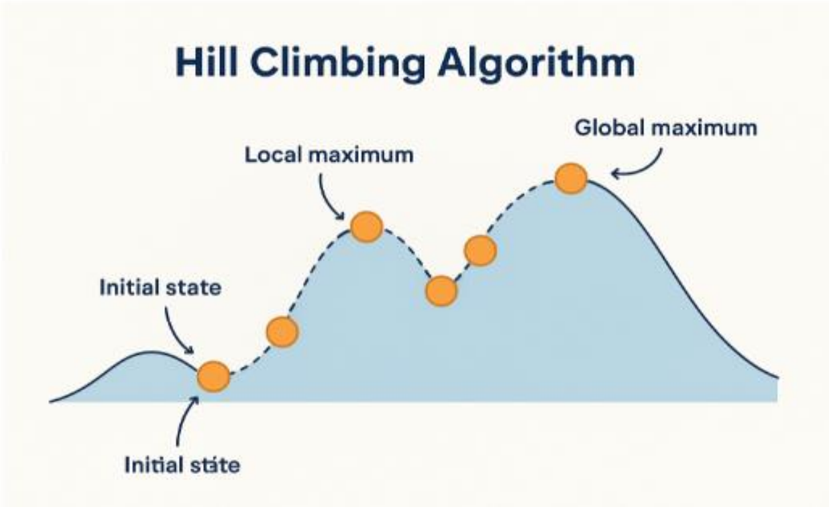
(Heuristic Algorithms)

Hill Climbing

Why Local Search is a heuristic:

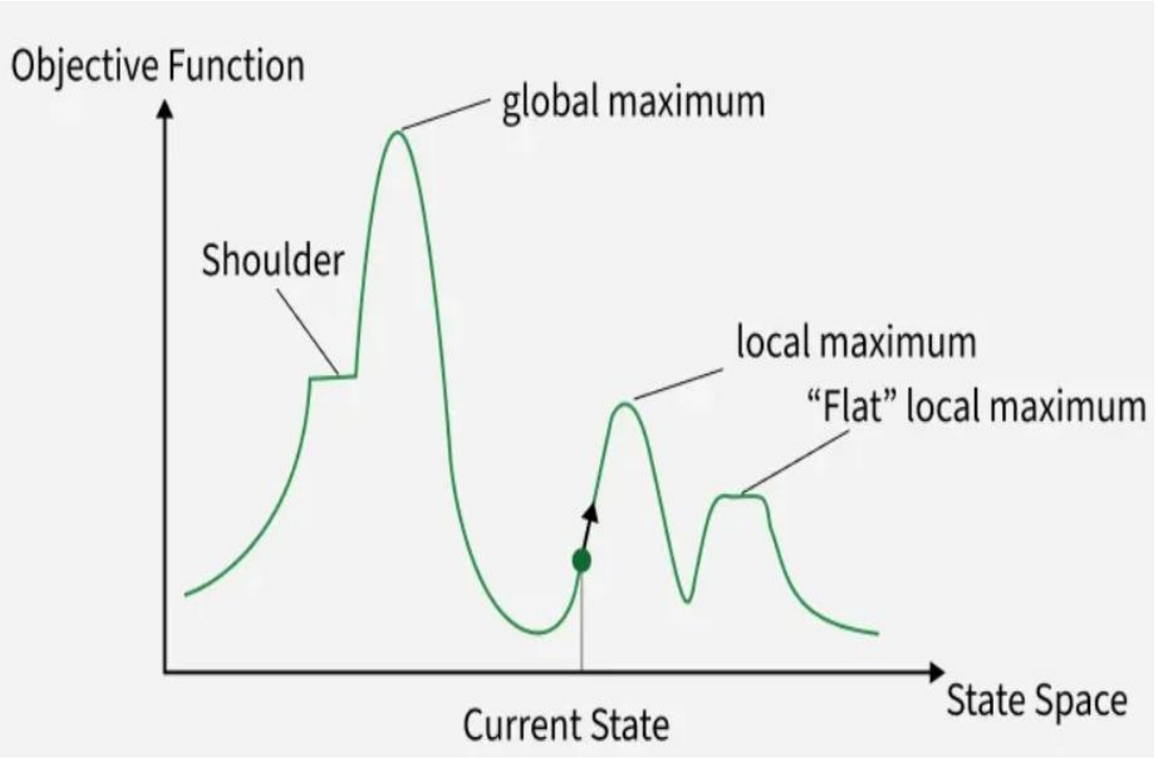
- Works with one solution at a time
- Greedy: always moves to the **best neighbor**.
- Fast and simple, but **problem-specific**.
- Often gets stuck in local optima. This means that the algorithm may stop at a solution that looks best in its neighborhood, but is not the best overall solution.
- Hill Climbing is a local search algorithm that iteratively moves from the current solution to a better neighboring solution. For maximization, it moves toward higher objective values and may get trapped at a local maximum; for minimization, it moves toward lower objective values and may get trapped at a local minimum.





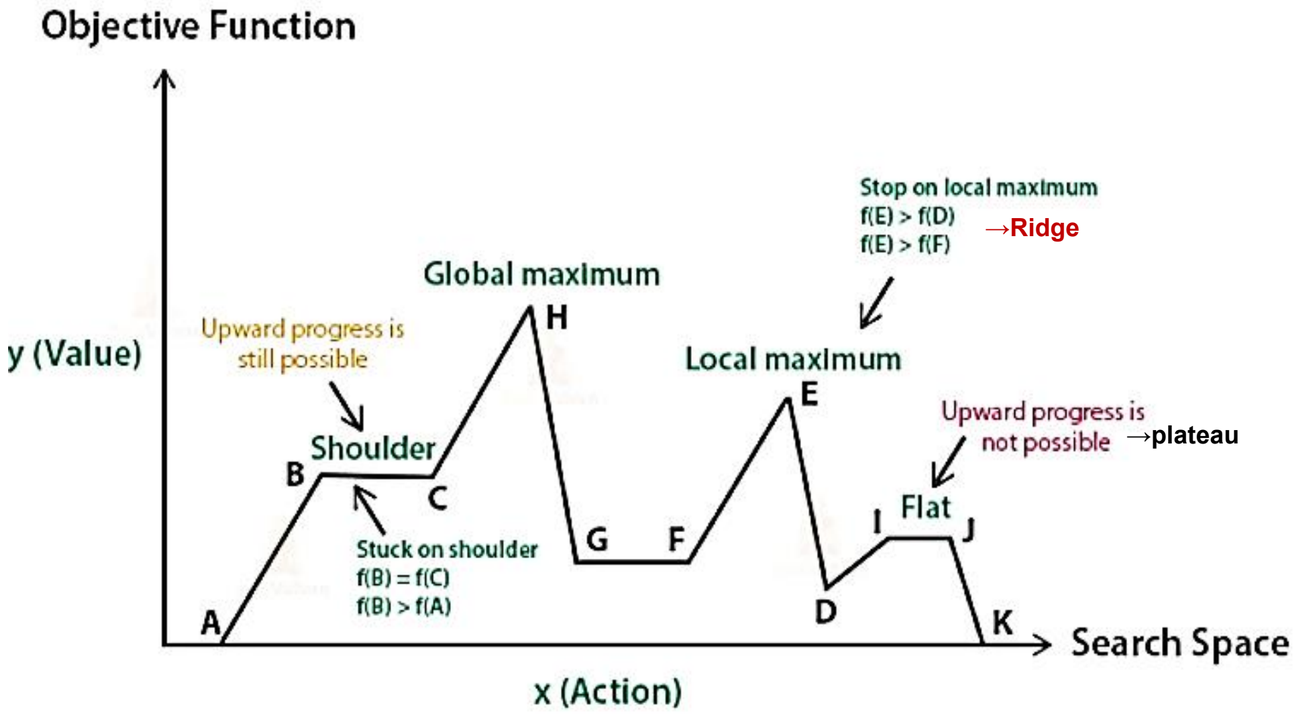
State-Space Diagram in Hill Climbing

- State-space diagram is a visual representation of all possible states the search algorithm can reach, plotted against the values of the objective function (the function we aim to maximize).
- The optimal solution in the state-space diagram is represented by the state where the **objective function** reaches its maximum value, also known as the **global maximum**.
- In the state-space diagram:
- **X-axis**: Represents the state space which includes all the possible states or configurations that the algorithm can reach.
- **Y-axis**: Represents the values of the objective function corresponding to each state.



Analogy of State-Space Diagram

1. **Local Maximum**: A local maximum is a state better than its neighbors but not the best overall. While its objective function value is higher than nearby states, a global maximum may still exist.
2. **Global Maximum**: The global maximum is the best state in the state-space diagram where the objective function achieves its highest value. This is the optimal solution the algorithm seeks.
3. **Current State**: The current state refers to the algorithm's position in the state-space diagram during its search for the optimal solution.
4. **Plateau/Flat:** A plateau is a flat region where neighboring states have the same objective function value, making it difficult for the algorithm to decide on the best direction to move.
5. **Ridge**: A ridge is a higher region with a slope which can look like a peak. This may cause the algorithm to stop prematurely, missing better solutions nearby.
6. **Shoulder**: A shoulder is a plateau with an uphill edge allowing the algorithm to move toward better solutions if it continues searching beyond the plateau.



Failure Cases in Hill Climbing: Plateaus, Ridges & Local Minima

Failure Case	Cause	Effect on Search	Fix / Improvement
Plateaus	Flat region with equal neighbor values	Algorithm stops early (no direction to move)	Random restarts, add noise
Ridges	Gradient exists but only along diagonal directions	Slow zigzag progress	Use diagonal/variable neighborhood
Local Minima (false trap)	A local minimum is a point where all neighbors are worse, but it's not the global minimum.	Algorithm stuck in sub-optimal solution	Simulated annealing, tabu search

Neighborhood-based Local Search

What is a Neighborhood?

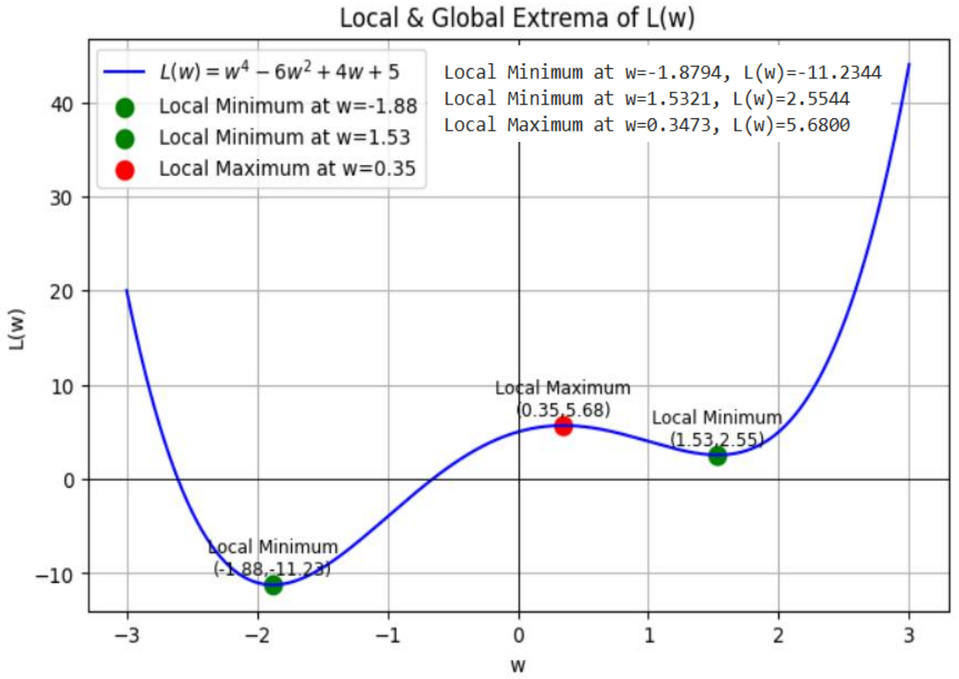
- In optimization, a neighborhood of a solution s is the set of candidate solutions you can "reach" by applying a small change (move) to s.
- Type of neighborhood depends on the nature of variables (discrete vs. continuous).

Types of Neighborhood

Neighborhood Type	Variable Category	Move / Perturbation	Simple Example
Additive	Continuous / numerical	Add or subtract a fixed step: $x' = x \pm \Delta$	$x = 5, \Delta = 0.1 \rightarrow\ 5.1\ or\ 4.9$
Multiplicative	Continuous / numerical	Multiply or divide by a factor: $x' = x(1 \pm \alpha)$	$x = 100, \alpha = 0.1 \rightarrow\ 110\ or\ 90$
Combinatorial	Discrete / permutation / binary	Swap, insert, delete, flip, or exchange elements	**Example: Feature Selection (MNIST)** Current: {f1, f2, f3, f7, f9, f12} • **Swap:** replace f7 with f15 • **Flip:** add/remove a feature, e.g., add f20 • **Swap pair:** exchange two selected features with two unselected features
Probabilistic	Discrete or continuous	▪ Generate a neighboring solution using a probability distribution ▪ Instead of evaluating all possible neighbors, randomly sample a small subset of candidates, which is useful when the search space is large.	▪ $x' = x + \epsilon, \epsilon \sim N(0, \sigma^2)$ ▪ **Example:** CNN Hyperparameter Tuning Current: (LR = 0.01, Dropout = 0.3) Randomly sample a few neighbors, e.g., (0.02, 0.35) and (0.005, 0.25). → Move to an improved neighbor; otherwise, sample again.

Example (1):

- Assume the loss function in simple Neural network model is: $L(w) = w^4 - 6w^2 + 4w + 5$
- This is a non convex function



(i) From mathematical point of view, we compute critical points first.

- First Derivative: $\frac{dL}{dw} = 4w^3 - 12w + 4$
- Critical points: $4w^3 - 12w + 4 = 0$

$$w = -1.88, 0.35, 1.53$$

- Second derivative: $L'' = 12w^2 - 12$

$w = -1.88$	$L'' > 0\ (global\ minimum)$
$w = 0.35$	$L'' < 0\ (local\ maximum)(hill)$
$w = 1.58$	$L'' > 0\ (local\ minimum)$

(ii) From AI/engineering practice point of view:

- In AI/engineering practice, we don't know them, we just start somewhere and see where local search takes us.
- Assume initially $w = 0$
- Now we will use both gradient descent and hill climbing

1. Using Gradient descent

- Gradient descent update rule: $w_{t+1} = w_t - \eta \frac{dL}{dw}$ (assume η=0.1)

$w_{t+1} = w_t + \Delta, where\ \Delta = -\eta \frac{dL}{dw}$

$L(w) = w^4 - 6w^2 + 4w + 5$

Start with: $w_0 = 0,\ then\ loss \quad L(0) = 5$

$\frac{dL}{dw} = L'(w) = 4w^3 - 12w + 4$

$w_{t+1} = w_t - \eta \frac{dL}{dw} = 0 - 0.1 \times 4 = -0.4$

- **Positive gradient** → Move w **left (–)**
- **Negative gradient** → Move w **right (+)**
- The size of the move depends on **gradient × learning rate**.
- Here, η=0.1 is causing **overshooting and oscillation**, so the algorithm doesn't settle quickly at the minimum.

Iteration	w	$L(w)$	Gradient→ $L'(w)$	$New\ w$	Move
0	0.000	5.000	4.000	-0.400	-0.400
1	-0.400	2.466	8.544	-1.254	-1.254
2	-1.254	-6.983	11.158	-2.370	-2.370
3	-2.370	-6.629	-20.817	-0.288	-0.288
4	-0.288	3.354	7.366	-1.025	-1.025
5	-1.025	-4.301	11.992	-2.224	-2.224
6	-2.224	-9.104	-13.328	-0.891	-0.891
7	-0.891	-2.703	11.864	-2.078	-2.078
8	-2.078	-10.576	-6.951	-1.383	-1.383

Conclusion: The designer may need to **choose an appropriate learning rate and increase the number of iterations** to allow Gradient Descent to converge.

2. Using Hill climbing

- $L(w) = w^4 - 6w^2 + 4w + 5$
- **Assumptions:** Start $w = 0$, $Step\ Size\ = 1$, $Neighbors\ = \{w - 1, w + 1\}$

Iteration	Current w	L(w)	Neighbors	Move
0	0	5	$-1 \rightarrow L(-1) = -4$, $+1 \rightarrow L(1) = 4$	$\rightarrow\ -1$ Both neighbors are improvements, but −1is selected because it gives the lowest objective value.
1	-1	-4	$-2: -11$, $0: 5$	$\rightarrow\ -2$
2	-2	-11	$-3: 20$, $-1: -4$	STOP

Summary: from the previous results we can conclude that:

1. In Hill Climbing:

Calculate neighbors → compare → Take only if better

2. In Gradient Descent:

- Calculate gradient → calculate direction/step → Take the calculated step
- So it accepts all values, unless you have explicitly added a separate acceptance rule.

Pros of local search	Cons of local search
▪ Simple and easy to implement. ▪ Works well if the function is smooth and unimodal. ▪ Low memory requirement. ▪ Useful for large-scale problems where exact search is impossible	▪ Highly dependent on the starting point → different starts gave different outcomes. ▪ Can get stuck in local minima and miss the global minimum. ▪ Requires careful choice of step size η to avoid overshooting or slow progress. ▪ Purely greedy → no memory of past moves, cannot escape bad traps

Another problem in gradient descent in convex functions

Convexity vs. Optimization Convergence

- **Convex function:** Has a single global minimum, e.g., $f(x) = (x - 3)^2$
- **Gradient Descent:** Can converge to the global minimum, but the convergence speed depends on the:

a) Learning rate,
a) Starting point,
b) Number of iterations.

- **Limited iterations:** If Gradient Descent does not reach the minimum within 10 iterations, this indicates slow convergence under the selected settings, not algorithm failure.
- Hill Climbing: May reach the optimum faster if its step strategy allows larger or more effective moves.
- Key distinction: Convexity guarantees the optimization landscape, not the speed of convergence.
- Fair comparison: When using a fixed iteration budget, compare the algorithms based on their solution quality and convergence speed within that budget.

We want to **minimize** the following **convex** function using gradient descent (assume η =0.1 and initial value $x = 0$):

$$f(x) = (x-3)^2$$

Gradient rule: $x_{new} = x_{old} - \eta \frac{dL}{dx}$

$$f(x) = (x-3)^2$$

$$\frac{dL}{dx} = 2(x-3)$$

$$x_{new} = x_{old} - 0.1 \times 2(x-3)$$

Step	x_{old}	$f(x)$	Gradient $[2(x-3)]$	Update $[x_{new}]$
1	0.0000	9.0000	-6.0000	0.6000
2	0.6000	5.7600	-4.8000	1.0800
3	1.0800	3.6864	-3.8400	1.4640
4	1.4640	2.3593	-3.0720	1.7712
5	1.7712	1.5099	-2.4576	2.0170
6	2.0170	0.9663	-1.9661	2.2136
7	2.2136	0.6184	-1.5728	2.3709
8	2.3709	0.3958	-1.2582	2.4967
9	2.4967	0.2533	-1.0066	2.5974
10	2.5974	0.1621	-0.8052	2.6779

Note:

- Change for loop: `for i in range(50):`
- Change error to be 0.1

```python
# minimize f(x) = (x - 3)^2

x = 0          # starting value
lr = 0.1       # step size

for i in range(10):
    f = (x - 3)**2            # compute function
    gradient = 2 * (x - 3)    # compute derivative

    x = x - lr * gradient     # update x

    print("Step", i+1, "x =", round(x, 4), "f(x) =", round(f, 4))

# final result
print("\nFinal x =", round(x, 4))

# check step
if abs(x - 3) < 0.01:
    print("Correct solution ✅")
else:
    print("Not accurate ❌")
```

```
Step 1 x = 0.6 f(x) = 9
Step 2 x = 1.08 f(x) = 5.76
Step 3 x = 1.464 f(x) = 3.6864
Step 4 x = 1.7712 f(x) = 2.3593
Step 5 x = 2.017 f(x) = 1.5099
Step 6 x = 2.2136 f(x) = 0.9664
Step 7 x = 2.3709 f(x) = 0.6185
Step 8 x = 2.4967 f(x) = 0.3958
Step 9 x = 2.5973 f(x) = 0.2533
Step 10 x = 2.6779 f(x) = 0.1621

Final x = 2.6779
Not accurate ❌
```

Now, solve the previous example with hill climbing:

Minimize the function using **Hill Climbing algorithm**:$f(x) = (x-3)^2$ given that:

- Initial solution: $x = 0$
- Neighbor: $x \pm 1$
- Selection rule: choose the **best (minimum) neighbor**
- Stop when no improvement is possible or optimal point is reached

Step	Current (x)	f(x)	Neighbor Left	Neighbor Right	Best Neighbor	Best Value	Decision	New x
0	0	9	-1 → 16	1 → 4	1	4	Move	1
1	1	4	0 → 9	2 → 1	2	1	Move	2
2	2	1	1 → 4	3 → 0	3	0	Move	3
3	3	0	2 → 1	4 → 1	—	—	Stop	3

- **Repeat your solution if we change initial value : $x = 6$**

Step	Current (x)	(f(x))	Neighbor Left	Neighbor Right	Best Neighbor	Best Value	Decision	New x
1	6	9	5 → 4	7 → 16	5	4	Move	5
2	5	4	4 → 1	6 → 9	4	1	Move	4
3	4	1	3 → 0	5 → 4	3	0	Move	3
4	3	0	2 → 1	4 → 1	—	—	Stop (optimal)	3

Conclusion

1. For a simple convex function, we applied both Gradient Descent and Hill Climbing, starting from $x = 0$.
2. With Gradient Descent ($\eta = 0.1$), the algorithm does not reach the optimum within 10 iterations; however, it continues moving toward the optimum and may converge after more iterations.
3. In contrast, Hill Climbing reaches the optimum in only 3 iterations. Interestingly, even when starting from $x = 6$, Hill Climbing reaches the optimum in 3 iterations.
4. Therefore, even for a simple convex function, the optimization method and its parameters matter. Gradient Descent may require many iterations depending on the learning rate and stopping criterion.
5. Now consider a non-convex function: the problem becomes more challenging because there may be multiple local minima, maxima, and saddle points. A local method can become trapped in a poor solution.
6. This is the motivation for studying metaheuristic optimization methods in this course: they are designed to perform broader exploration of the search space and reduce the risk of becoming trapped in a single local optimum.

Final pseudo code and mathematical representation of the Neighborhood-based Local Search algorithm

```
1:  x = GenerateInitialSolution()              // Initialize
2:  repeat
3:      Neighborhood N = Generate Neighbors(x)        // Define candidate moves
4:      x′ = Find Improving Neighbor(N)       //Find a better solution in the neighborhood
5:      if f(x′) < f(x) then       //If an improving  neighbor exists …
6:      x = x′       // ⋯ move to it(Descent)
7:      else
8:          break       // ⋯ else, terminate(LocalOptimum)
9:      end if
10: until termination condition met
```

$$x^{(t+1)} = \arg\min_{z \in N(x^{(t)})} f(z)$$

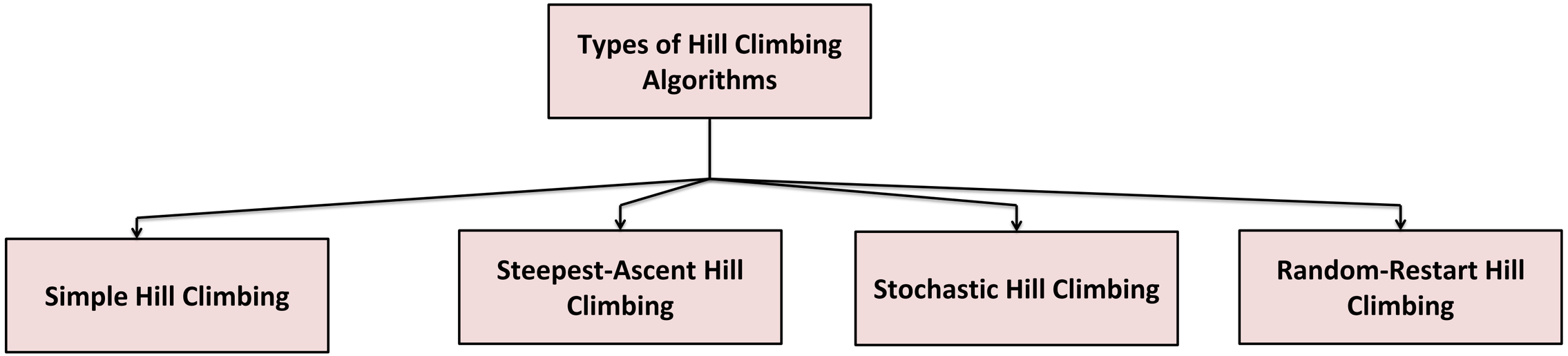
$$x^{(t+1)} = \begin{cases} x' \in N(x^{(t)}) & \text{if } \exists x' \in N(x^{(t)}) : f(x') < f(x^{(t)}) \\ x^{(t)} & \text{otherwise (terminate)} \end{cases}$$

Where:

- **Solution** (x): A candidate answer to the optimization problem.
- **Fitness Function** $(f(x))$: A function that measures the quality of a solution (lower is better for minimization).
- **Neighborhood** $(N(x))$: The set of all solutions that can be reached from x by a predefined "move" (e.g., swapping two elements, changing a variable slightly).
- **Local Optimum**: A solution x where no neighbor has a better fitness value, i.e.,$f(x') \geq f(x)\ for\ all\ x'\ in\ N(x)$.

Types of Hill Climbing Algorithms

Hill Climbing itself has **variants**, which improve performance depending on the problem:



Types of Hill Climbing Algorithms

Type	How it chooses the next move	Main idea	Advantage	Limitation
Simple Hill Climbing	Selects the first neighbor that improves the current solution	Move to the first better solution found	Simple and fast	May miss a better neighbor
Steepest-Ascent Hill Climbing	Evaluates all neighbors and selects the best improving neighbor	Always make the largest improvement	Usually finds better moves locally	More computationally expensive
Stochastic Hill Climbing	Randomly selects an improving neighbor, often with preference toward better moves	Introduces randomness into the search	Less deterministic; can explore different paths	May choose a weaker improvement
Random-Restart Hill Climbing	Runs Hill Climbing multiple times from different random starting points	Escape local optima by restarting	Higher chance of finding the global optimum	Requires multiple runs

Example: Suppose we want to **maximize** $f(x)$, and we can move from $x \; to \; x-1 \; or \; x+1$.

x	0	1	2	3	4	5	6	7	8	9	10
$f(x)$	2	5	8	10	7	6	8	11	15	12	9

There are **two peaks**:

- Local maximum: $x = 3, f(x) = 10$
- Global maximum: $x = 8, f(x) = 15$

Assume we start at $\boldsymbol{x = 2}$.

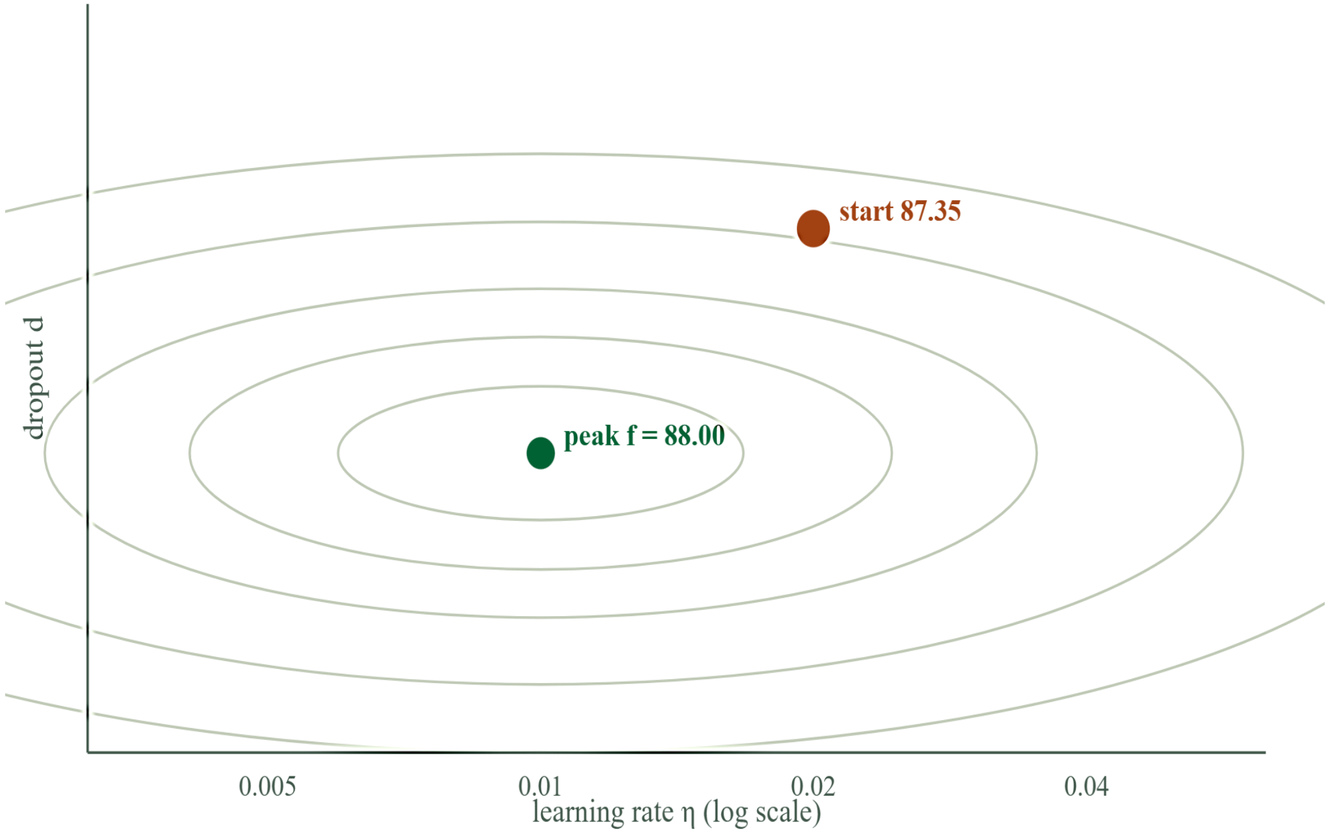
Step	**Simple HC** I found a better neighbor → move there"	**Steepest-Ascent HC** Let me check all neighbors → choose the best one	**Stochastic HC** There are better neighbors → randomly choose one	**Random-Restart HC** I got stuck → start the whole search again somewhere else
Start	$x = 2, f = 8$	$x = 2, f = 8$	$x = 2, f = 8$	$Random\ start\ x = 2$
1	▪ $Neighbors$: $1(5), 3(10) \rightarrow 3$ ▪ If he found 1 better from the beginning, then it will not try 3	▪ $Neighbors$: $1(5), 3(10) \rightarrow 3$ ▪ $Best\ neighbor \rightarrow 3$	$Choose\ improving$: $neighbor \rightarrow 3$	Check 1(5), 3(10) → 3
2	$Neighbors$: $2(8), 4(7) \rightarrow STOP$	$Best\ neighbor$: $2(8), 4(7) \rightarrow STOP$	$No\ improving\ neighbor \rightarrow STOP$	At x=3: 2→8, 4→7 → STOP
3	$Final$: $x = 3, f = 10$	$Final$: $x = 3, f = 10$	$Final$: $x = 3, f = 10$	Restart: x=6 Check $5(6), 7(11) \rightarrow$ Move to 7
4	—	—	—	At x=7: check 6(8), 8(15)→ Move to 8
5	—	—	—	At x=8 : check 7(11), 9(12)→ STOP
Final	$Local\ \max = 10$	$Local\ \max = 10$	$Local\ \max = 10$	$Global\ max = 15$

Example: two dimensional space

Assume we need to optimize both learning rate and dropout in order to maximize: $f(\eta, d) = 88 - (\log_{10}(\eta) + 2)^2 - 25(d - 0.4)^2$

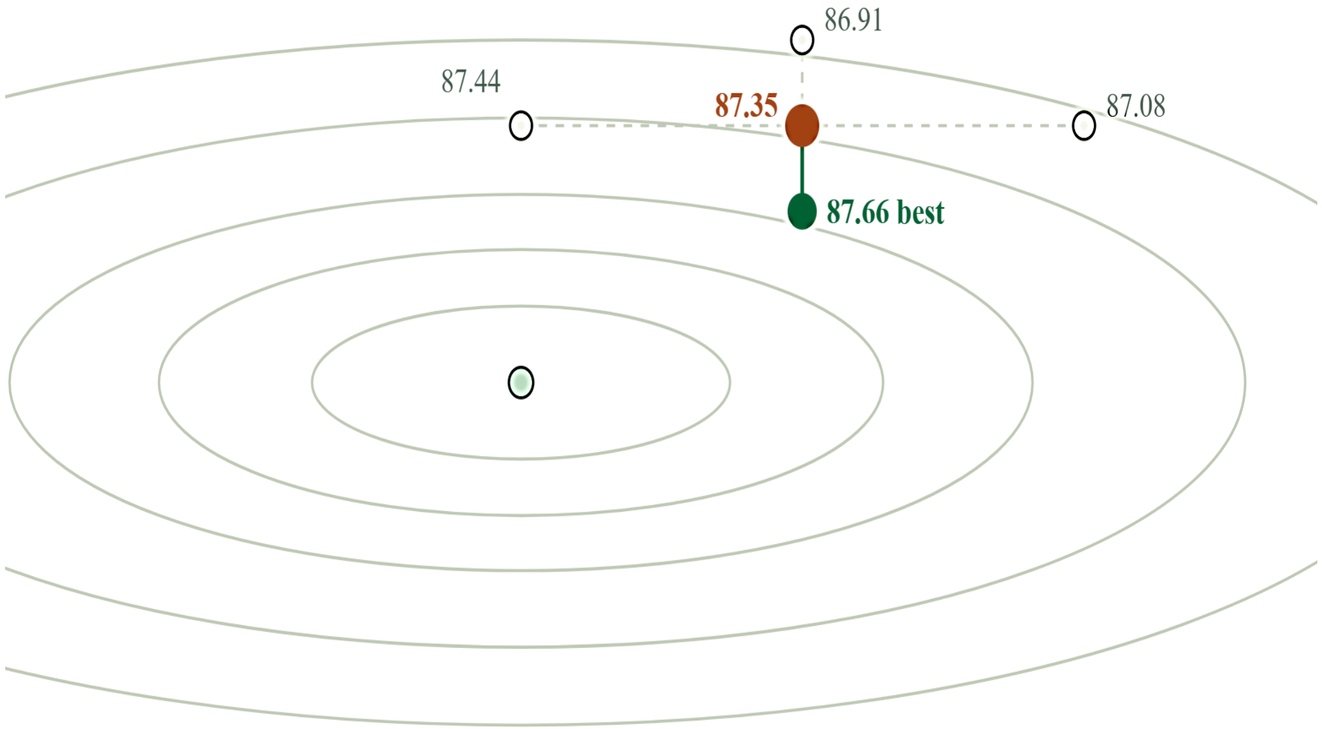
Given that:

- η moves multiplicatively: $\eta \times 2$ *or* $\eta/2$
- d moves additively: $d \pm 0.05$
- Four neighbors are evaluated each iteration.
- Initial values: $\eta = 0.02, d = 0.55 \rightarrow f = 87.35$
- Expected maximum $\eta = 0.01, d = 0.40 \rightarrow f = 88$ (from mathematics)
- Assume 5 iterations



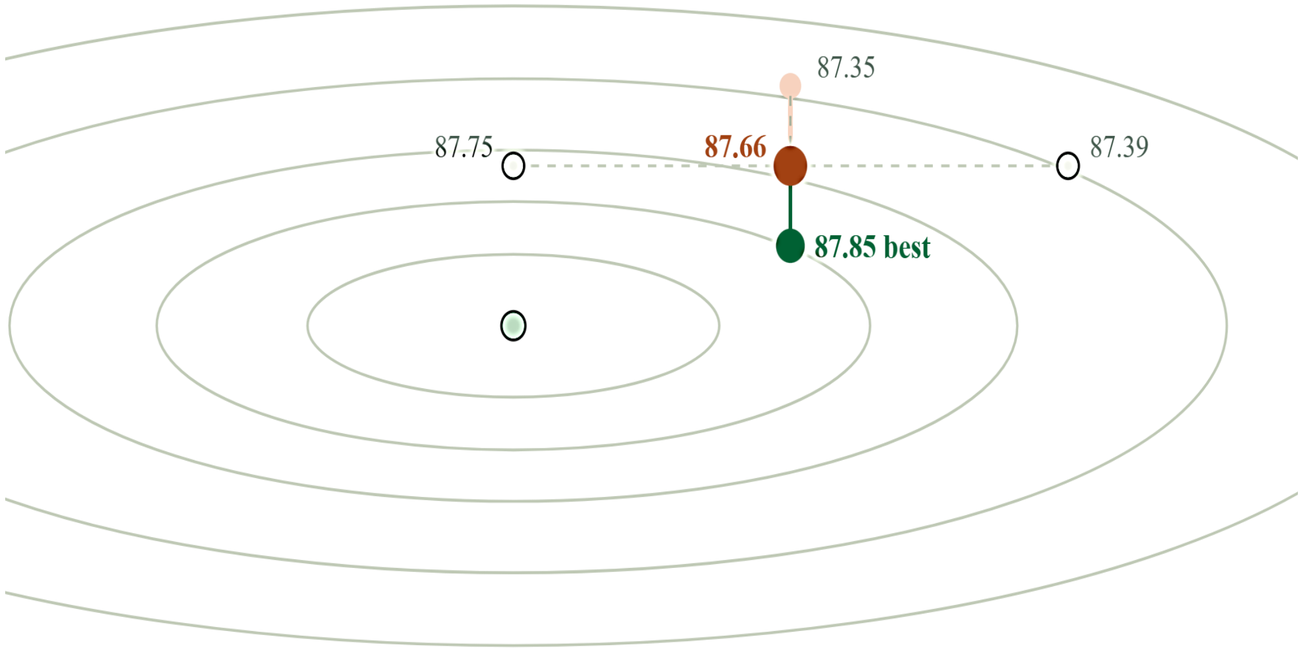
Iteration (1): Evaluate every neighbor of (0.02, 0.55)

$Neighbors\ of\ (0.02, 0.55)$	$f(\eta, d)$
$(0.01, 0.55)$	87.44
$(0.04, 0.55)$	87.08
$\mathbf{(0.02, 0.50)}$	$\mathbf{87.66 \leftarrow best}$
$(0.02, 0.60)$	86.91



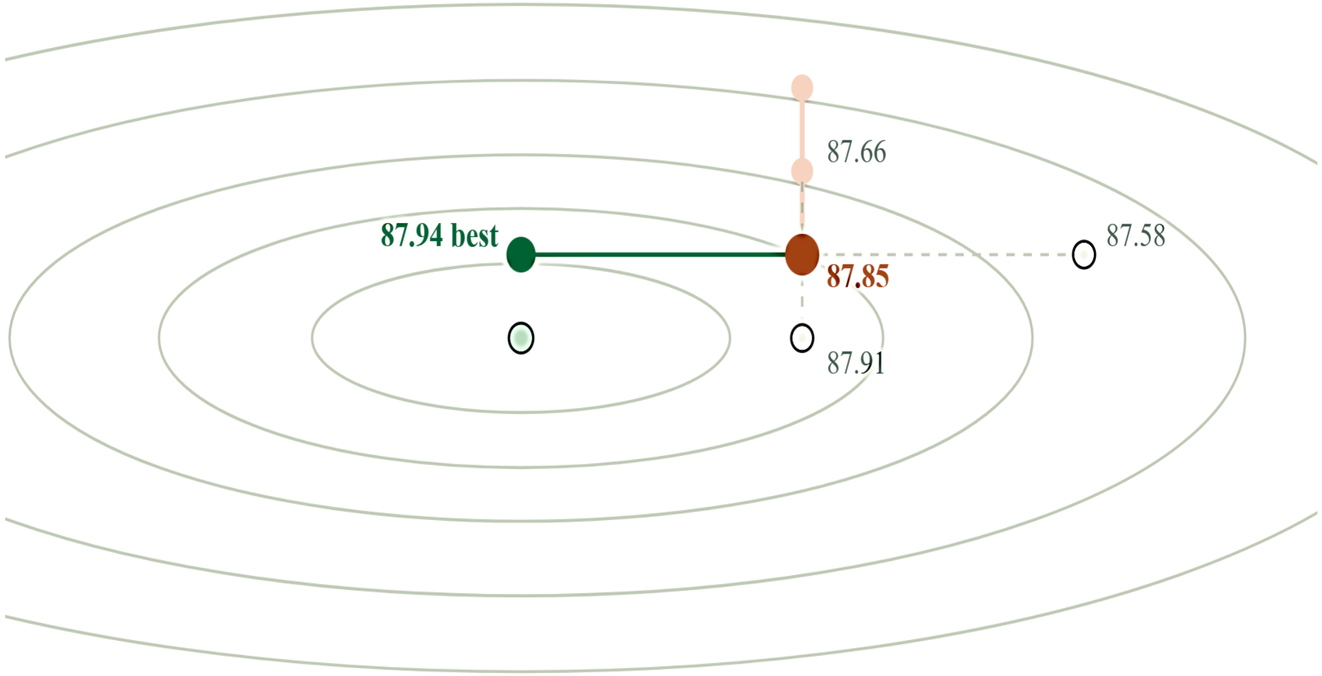
Iteration (2): Evaluate every neighbor of (0.02, 0.50)

$Neighbors\ of\ (0.02, 0.50)$	$f(\eta, d)$
$(0.01, 0.50)$	87.75
$(0.04, 0.50)$	87.39
$\mathbf{(0.02, 0.45)}$	$\mathbf{87.85} \leftarrow \boldsymbol{best}$
$(0.02, 0.55)$	87.35



Iteration (3): Evaluate every neighbor of (0.02, 0.45)

$Neighbors\ of\ (0.02, 0.45)$	$f(\eta, d)$
$(0.01, 0.45)$	$\mathbf{87.94} \leftarrow \boldsymbol{best}$
$(0.04, 0.45)$	87.58
$(\mathbf{0.02}, \mathbf{0.40})$	87.91
$(0.02, 0.50)$	87.66

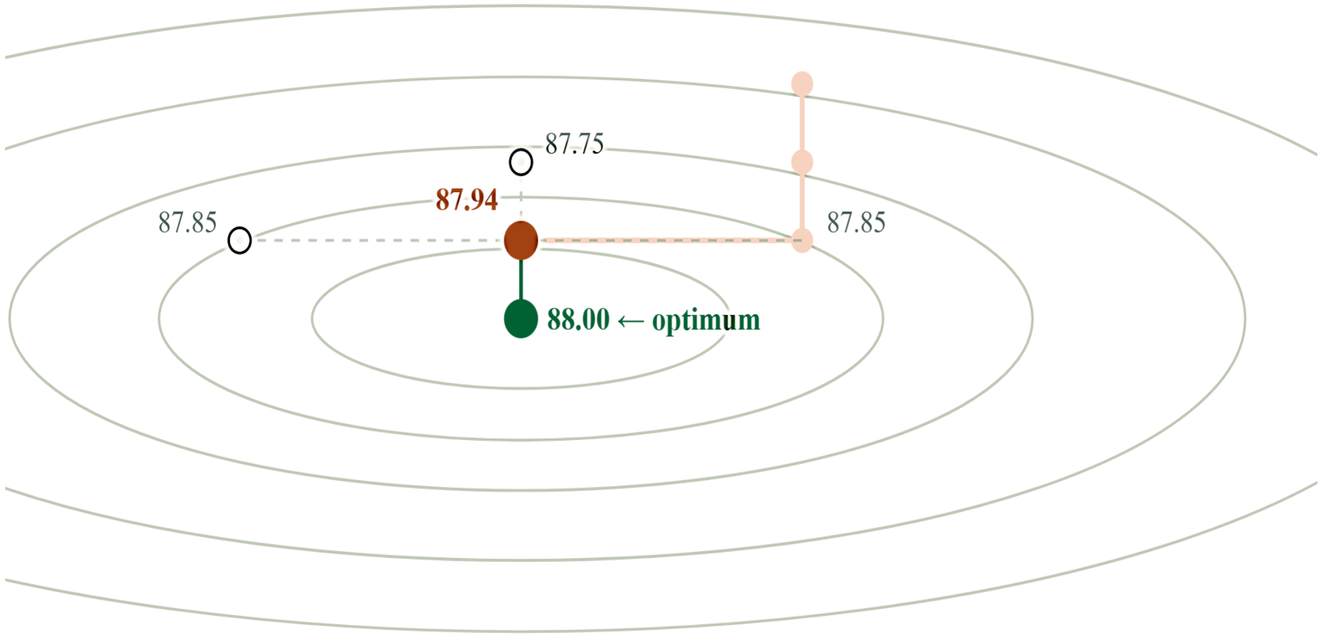


Iteration (4): Evaluate every neighbor of (0.02, 0.45)

Note that:

- Two neighbors tie at 87.85 on either side of the current point
- The classic signature of sitting on a ridge along one axis while still climbing the other.

$\boldsymbol{Neighbors\ of\ (0.01, 0.45)}$	$\boldsymbol{f(\eta, d)}$
$(0.005, 0.45)$	87.85
$(0.02, 0.45)$	87.85
$\boldsymbol{(0.01, 0.40)}$	$88.00 \leftarrow \boldsymbol{best}$
$(0.01, 0.50)$	87.75

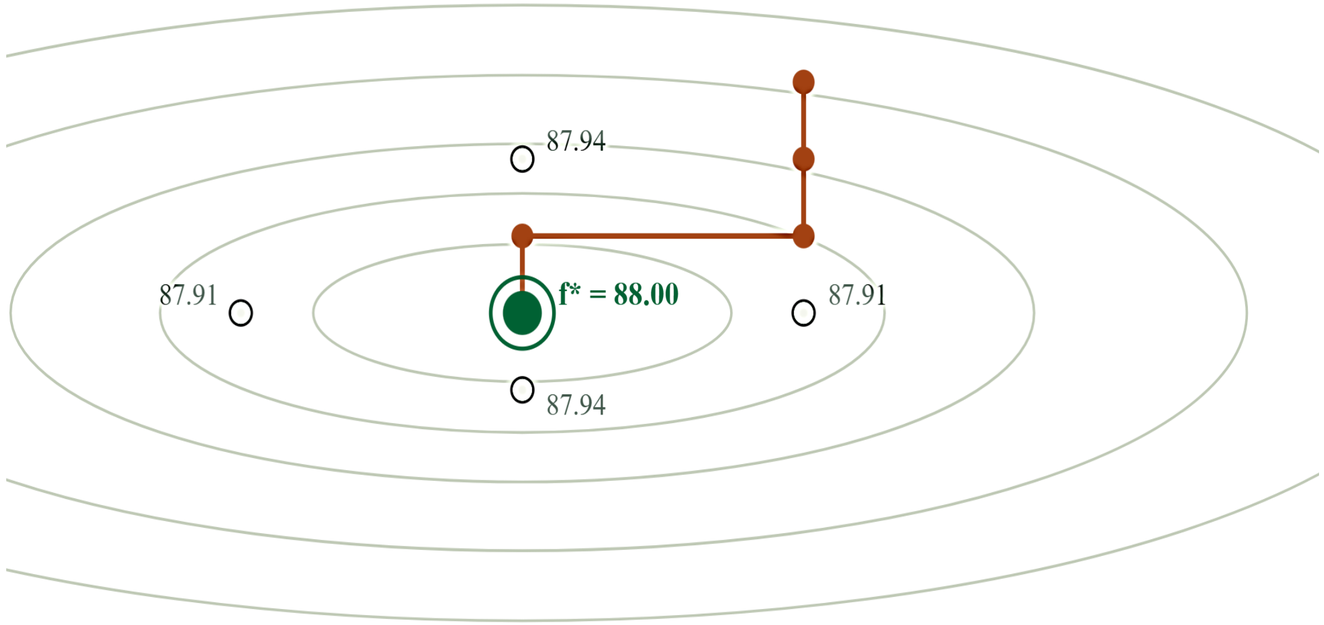


Iteration (5): Evaluate every neighbor of (0.01, 0.40)

No neighbor improves → terminate as from previous iteration we get:

$f(\eta, d) = 88 - (\log^{10}(0.01) + 2)^2 - 25(0.4 - 0.4)^2 = 88$

$Neighbors\ of\ (0.01, 0.40)$	$f(\eta, d)$
$(0.005, 0.40)$	87.91
$(0.02, 0.40)$	87.91
$(0.01, 0.35)$	87.94
$(0.01, 0.45)$	87.94



87.35 (iter 0) → **87.66** (1) → **87.85** (2) → **87.94** (3) → **88.00** (4)

Stage 1:

```
import tensorflow as tf
from tensorflow.keras import layers, models, optimizers
from tensorflow.keras.datasets import mnist
import numpy as np

# Load MNIST data
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0
x_train = x_train[..., np.newaxis]
x_test = x_test[..., np.newaxis]

# Build a simple CNN model
def build_model(eta, d):
    model = models.Sequential([
        layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
        layers.MaxPooling2D((2,2)),
        layers.Dropout(d),
        layers.Flatten(),
        layers.Dense(64, activation='relu'),
        layers.Dropout(d),
        layers.Dense(10, activation='softmax')
    ])
    opt = optimizers.Adam(learning_rate=eta)
    model.compile(optimizer=opt, loss='sparse_categorical_crossentropy', metrics=['accuracy'])
    return model
```

Stage 2:

```
# Evaluate model performance (validation accuracy)
def evaluate(eta, d, epochs=2):
    model = build_model(eta, d)
    history = model.fit(x_train, y_train, epochs=epochs, batch_size=128,
                        validation_split=0.2, verbose=0)
    val_acc = history.history['val_accuracy'][-1]
    return val_acc
```

Stage 3: The optimization part

```
# Hill climbing algorithm
def hill_climb(initial_eta=0.001, initial_d=0.3, max_iters=5):
    eta, d = initial_eta, initial_d
    best_score = evaluate(eta, d)
    print(f"Initial: eta={eta}, d={d}, ValAcc={best_score:.4f}")
    for it in range(max_iters):
        # Define neighborhood
        neighbors = [
            (eta * 2, d), (eta / 2, d),
            (eta, d + 0.1), (eta, d - 0.1)
        ]
        # Keep valid neighbors only
        neighbors = [(e, dr) for e, dr in neighbors if e > 0 and 0 < dr < 1]
        # Evaluate neighbors
        scores = [(evaluate(e, dr), e, dr) for e, dr in neighbors]
        # Pick best neighbor
        best_neighbor = max(scores, key=lambda x: x[0], default=(best_score, eta, d))
        if best_neighbor[0] > best_score:  # improvement
            best_score, eta, d = best_neighbor
            print(f"Iter {it+1}: eta={eta}, d={d}, ValAcc={best_score:.4f}")
        else:
            print("No better neighbor found. Stopping.")
            break
    print(f"\nFinal solution: eta={eta}, d={d}, ValAcc={best_score:.4f}")
    return eta, d, best_score
# Run hill climbing
hill_climb()
```

Results:

```
Initial: eta=0.001, d=0.3, ValAcc=0.9768
Iter 1: eta=0.002, d=0.3, ValAcc=0.9816
No better neighbor found. Stopping.

Final solution: eta=0.002, d=0.3, ValAcc=0.9816
(0.002, 0.3, 0.9815833568572998)
```